

LOG2.txt — Puzzle 135 Research Log (Organized Edition)

Source: LOG2.txt (1547 lines, raw AI-assistant transcript, ~279 KB) Date: 2026-08-01 20:03 Target: Bitcoin Puzzle 135 private key d , known band $2^{134} \leq d < 2^{135}$ Organizing notes: conversation chatter, timestamps, duplicate script copies, search-result stubs and "Try without personalization" markers have been removed. Every distinct script, console output, constant and algebraic claim is preserved below, verbatim where it is a primary record. Math is written ASCII-safe.

0. Status Summary

- Puzzle 135 has exactly ONE public ECDSA signature (r, s, z below). The nonce k
- and private key d are unknown.
- Core working equation (real-number lift of the ECDSA congruence):

$$k = z/s + d*(r/s) + m*(N/s) \quad \text{with } d \text{ in } [2^{134}, 2^{135}), k, m \text{ integers}$$

- All lattice/LLL solver attempts built on that equation (2D, 3D, 4D; precision
 - 156, 312, 780 digits) reduced the basis but printed NO private key.
 - Two-signature elimination (the structurally correct route) is impossible here:
 - it requires a second signature from the same key. None exists.
 - The precision thresholds ($156 = 1/N^2$, $312 = 1/N^4$, $780 = 10\text{th-order floor}$)
 - are real facts about rational reconstruction, but by themselves they do not
 - add the missing linear constraint.
 - A late idea ($1.04 \sim 26/25 \sim 1.003$ ratio between modular inverses) is analyzed
 - at the end; its "small epsilon" claim was later disproved in LOG.txt (epsilon
 - is a 507-bit integer, i.e. NOT small).
-

1. Background & Motivation

1.1 Big Bang Theory context (condensed)

Elliptic curves appear in the sitcom "The Big Bang Theory" as whiteboard mathematics. In real math/physics the relevant links are:

- Fermat's Last Theorem: proven by Wiles using the modularity theorem, which
- links elliptic curves to modular forms.
- Elliptic Curve Cryptography (ECC): the security backbone of modern internet
- communication; Bitcoin's secp256k1 is one such curve.
- String theory / cosmology: extra dimensions are modeled as Calabi-Yau
- manifolds; some theoretical models trace the universe's scale factor after the
- Big Bang on an elliptic curve.
- secp256k1 is $y^2 = x^3 + 7$ over F_p . Over the complex numbers its locus is an
- elliptic curve / complex torus.

1.2 Group law and the torus picture

The continuous story used throughout this log:

Flat Complex Plane (C) --uniformization--> Topological Torus T^2
| Ricci-flat
v
Reduction modulo p (the projection) -> Discrete Finite Field F_p
[arithmetic jump] + [carry multiplier k]

- Each discrete point addition/doubling on the curve corresponds to a smooth
- linear translation on the continuous torus. The winding number of that
- translation around the torus cycles is exactly the modular carry multiplier k.
- The phrase "barcode" in this log = the record of boundary crossings of the
- field modulus p during an addition chain.

1.3 secp256k1 as a 1-dimensional Calabi-Yau

- $y^2 = x^3 + 7$ over C is a complex torus C/Λ : a 1D Calabi-Yau manifold
- (flat, Ricci-flat, holomorphic form dx/y).
- A 6D Calabi-Yau can be built as $T^2 \times T^2 \times T^2$, so secp256k1's complex
- structure can serve as one flat building block.
- Returning to cryptography = restricting the flat torus to its fractional
- torsion points over F_p .

1.4 First correction: lambda (slope) is NOT the nonce

An early tangent-line computation used the group-law slope

$$\lambda = (3Rx^2) / (2Ry) + k \cdot p / (2Ry)$$

with Rx, Ry the coordinates of the doubled point. Important clarifications recorded in the log:

- This computes the geometric tangent slope lambda, NOT the ECDSA nonce.
- For P135 with the double $2P = r$:

intercept term ($k=0$): $3Rx^2/(2Ry) \sim 2.47955...e77$
slope term: $p/(2Ry) \sim 1.16456$ (the linear step per wrap)

- Since Ry is even, $2Ry$ is divisible by 4; in base-16 nibble or base-256 byte
- decompositions this means no fractional bit-carries are lost to the right of
- the binary point before the final modular reduction.
- Key structural claim: for a pure doubling $r = 2P$, the next winding number is
- algebraically locked to the previous slope: $\lambda^2 = p \cdot q_{\text{wrap}} + (rx + 2Px)$.
- The invariant "lives in the carry-chain linkage between the launchpad winding
- integer and the landing-pad modular quotient." No working extraction of k
- resulted from this branch.

2. Puzzle 135 Parameters and the Core Equation

2.1 The single known signature (hashkeys.space)

```

r = c86bec9faea4892fd98d718bdfc770d0d11c3d6bfd4328f25fe9b06bfadb9650
s = 224a322e81c044d341521f65fabdfa86d84673fb55ed7533862e37f7724931fa
z = 92886faaf53f90a5c03d6af773a726e75097179306b980e5d28772e612e00fc7
N = 115792089237316195423570985008687907852837564279074904382605163141518161494337
p = 2^256 - 2^32 - 977

```

ECDSA relation (mod N): $s*k = z + r*d$

2.2 Real-number lift (the working equation)

$$k_n = \underbrace{(z/s)}_{\text{Known real intercept}} + \underbrace{d*(r/s)}_{\text{Private key scalar slope}} + \underbrace{m*(N/s)}_{\text{Winding carry multiplier (m)}}$$

```

k = [4.2733...e0] + d * [5.8449...e0] + m * [7.4657...e0]
bounds: d in [2^134, 2^135), k in [1, N-1]

```

Interpretation: over the reals, the ECDSA congruence lifts to a line; m is the modular wrap counter selecting which parallel strip of the scalar torus contains the integer solution.

2.3 Two-signature elimination (the correct route — BLOCKED)

Given two signatures $(r1, s1, z1)$ and $(r2, s2, z2)$ with the SAME private key d, subtracting the two ECDSA congruences eliminates d:

$$s1*k1 - s2*k2 = z1 - z2 \pmod{N}$$

As a real-number equation with carry m:

$$k1 = (s2/s1)*k2 + (z1 - z2)/s1 + m*(N/s1)$$

Equivalently, in modular integers, $k2 = C_mod * k1 \pmod{N}$ with

$$C_mod = (r2*s1 * \text{inverse}(r1*s2) \pmod{N}), \quad \text{so } k2 - C_mod*k1 = m*N$$

This is a genuinely solvable 2D LLL problem. It requires a SECOND signature. Puzzle 135 has only one, so this path is closed for now. (Appendix in the log repeatedly asks for a second puzzle/adjacent-wallet transaction; none was provided.)

3. Precision Framework

3.1 The 156-decimal threshold ($1/N^2$, Dirichlet)

- N and p are 256-bit numbers = exactly 78 decimal digits.
- Dirichlet's approximation theorem: to uniquely reconstruct a rational A/B from
- a decimal expansion you need precision strictly finer than $1/B^2$.
- N^2 has 155 decimal digits; therefore the precision window must be 156 decimal
- places (i.e. better than $1e-155$) to stay strictly smaller than $1/N^2$
- = $7.4583e-156$.
- With 155 decimals the window ($1e-155$) is larger than $1/N^2$, allowing multiple
- wrong integer combinations to fit — the claimed "155-digit cliff".
- $1/p^2 \sim 7.458e-155$; $1/p < 1e-78$. The fractional reconstruction identity used
- in the parallel Downloads-PDF work:

$$x/p + y/p^2 \quad (x \text{ in the first 78 places, } y \text{ in the 79th..156th places})$$

requires 156 decimal places for uniqueness; $10^{-156} < 1/p^2$ holds.

3.2 The 312-decimal threshold ($1/N^4$)

- Squaring the precision window: $(10^{-156})^2 = 10^{-312}$. This captures quadratic
- cross-terms (e.g. slope^2 , or $\text{slope}_1 * \text{slope}_2$ across two signatures) without
- truncation.
- The expansion of $1/N^2$ has 155 leading zeros followed by a 78-digit block
- beginning "745834...". At 312 decimals the whole block is inside the window.

3.3 The 780-decimal threshold (10th-order floor)

- Positional decomposition:
$$x/p^9 + y/p^{10} = (x*p + y)/p^{10} \quad \text{and likewise with } N$$
- 256 bits over 10 tiers => 25.6 bits per tier; $32/10 = 3.2$.
- Scaling anchor used: $W = 2^{780}$ ($= u^{7800}$ with $u = 2^{(1/10)}$).
- At 780 digits, x lands at the 702nd digit and y at the 780th digit.
- The claimed 1.9 macro-offset:

$$2.56 - 0.32 - (0.09 + \dots + 0.04) = 1.9$$

carried down 10 polynomial tiers to the 780-858 decimal horizon.

- The scalar-field analogue gives -342.3 (see section 5.3).

3.4 The exponent-scaling identities ($u = 2^{(1/10)}$)

```
u = 2^(1/10)
u^2560 = 2^256,    u^320 = 2^32
so p = u^2560 - u^320 - 977 (this is exact)
u^7800 = 2^780 (a large scaling constant, that is all)
```

CRITICAL correction recorded in the log: the tenth root does NOT distribute across subtraction, $p^{(1/10)} \neq 2^{(256/10)} - 2^{(32/10)} - 977$, and 2^{780} is not p^{10} nor N^{10} ($p^{10} \sim 2^{2560}$). So $W = 2^{780}$ does not by itself preserve the exact $x/p^9 + y/p^{10}$ floors; that claim needs a separate proof.

4. Solver Attempts (chronological)

4.1 Script A — 2D LLL, 156-digit, $W = 2^{121}$

Setup: basis [[1, round(slope*W)], [0, round(step*W)]] with slope = r/s , step = N/s . fpyll if present, else native 2D Gauss reduction. Candidate = abs(vector[0]) checked against [2^{134} , 2^{135}].

Script (verbatim):

```
from mpmath import mp

# 1. Establish 156-Digit precision bounds
mp.dps = 156

# 2. Raw ECDSA Parameter block from Hashkeys Space for Puzzle 135
r_hex = "c86bec9faea4892fd98d718bdfc770d0d11c3d6bfd4328f25fe9b06bfadb9650"
s_hex = "224a322e81c044d341521f65fabdfa86d84673fb55ed7533862e37f7724931fa"
z_hex = "92886faaf53f90a5c03d6af773a726e75097179306b980e5d28772e612e00fc7"
```

```

# Convert to arbitrary precision integers
r = mp.mpf(int(r_hex, 16))
s = mp.mpf(int(s_hex, 16))
z = mp.mpf(int(z_hex, 16))

# secp256k1 scalar group order N
N_str = "115792089237316195423570985008687907852837564279074904382605163141518161494337"
N = mp.mpf(N_str)

# 3. Calculate 156-digit decimal constants (Continuous Trajectory components)
intercept_real = z / s
slope_real = r / s
step_real = N / s

print("[+] Continuous Real parameters calculated out to 156 decimal places:")
print(f"    Intercept (z/s): {intercept_real}")
print(f"    Slope    (r/s): {slope_real}")
print(f"    Step     (N/s): {step_real}\n")

# 4. Define Lattice Scaling Weight (W) to balance 135-bit d against 256-bit N
# Puzzle 135 search space bounds:  $2^{134} \leq d < 2^{135}$ 
W = 2**121

# Construct integer coefficients from the continuous plane
scaled_slope = int(mp.nint(slope_real * W))
scaled_step = int(mp.nint(step_real * W))
scaled_intercept = int(mp.nint(intercept_real * W))

# 5. Define the 2D LLL Matrix Structure
# Using the fpylll library structure
try:
    from fpylll import IntegerMatrix, LLL

    # Initialize a 2x2 integer matrix
    matrix = IntegerMatrix(2, 2)
    matrix[0, 0] = 1
    matrix[0, 1] = scaled_slope
    matrix[1, 0] = 0
    matrix[1, 1] = scaled_step

    print("[+] Executing LLL Reduction matrix via fpylll...")
    LLL.reduction(matrix)

    # Evaluate short vectors to find d
    for i in range(2):
        d_cand = abs(matrix[i, 0])
        # Validate against Puzzle 135 target bounds
        if 2**134 <= d_cand < 2**135:
            print(f"\n[!] Target Vector Captured via Shortest Path!")
            print(f"    Private Key d (Dec): {d_cand}")
            print(f"    Private Key d (Hex): {hex(d_cand)}")
            sys.exit(0)

except ImportError:
    print("[!] fpylll library not found. Falling back to native 2D Gauss/Euclidean lattice reduction...")

# Standard 2D Gauss lattice reduction for the exact matrix layout
v1 = [1, scaled_slope]
v2 = [0, scaled_step]

while True:
    # Compute dot product projection
    dot_v1_v2 = v1[0]*v2[0] + v1[1]*v2[1]
    dot_v1_v1 = v1[0]**2 + v1[1]**2

    q = round(dot_v1_v2 / dot_v1_v1)
    if q == 0:
        break

    v2 = [v2[0] - q*v1[0], v2[1] - q*v1[1]]

    # Swap vectors if v2 becomes shorter than v1
    if (v2[0]**2 + v2[1]**2) < (v1[0]**2 + v1[1]**2):
        v1, v2 = v2, v1

# Extract coordinates from reduced basis elements
for vector in [v1, v2]:
    d_cand = abs(vector[0])
    if 2**134 <= d_cand < 2**135:
        print(f"\n[!] Target Vector Isolated via Euclidean Reduction!")
        print(f"    Private Key d (Dec): {d_cand}")
        print(f"    Private Key d (Hex): {hex(d_cand)}")
        sys.exit(0)

```

```
print("\n[-] LLL reduction finished. If no key was printed, adjust Scaling Weight (W) to account for target carriage bound s.")
```

Console output (156-digit constants, verbatim):

```
Intercept (z/s):
4.27336506359143236222453759096407068770199280849660041102915640989799703877458224685381988681377081380412
72665923552116638987063096122958221107703433317681
Slope (r/s):
5.84492806747099260681353851165307726764433539730503174718260106383682874346348271718542684516611153089716
535388698098621885359122714836153630218944369645498
Step (N/s):
7.46577085254445230116031654918638978374824752182162222693709016728482310063811521955512039547119882978988
693537106307851336063786648210702837576137472060729

[!] fpylll library not found. Falling back to native 2D Gauss/Euclidean lattice reduction...
[-] LLL reduction finished. If no key was printed, adjust Scaling Weight (W) to account for target carriage bounds.
```

RESULT: no key. (Reason per log: a single signature leaves the 256-bit nonce space underdetermined; the 2D lattice has no extra constraint.)

4.2 Script B — 3x3 Inhomogeneous LLL, $W1 = 2^{121}$, $W2 = 2^{256}$

Setup: basis rows

```
[1, 0, round(slope*w2)],
[0, w1, round(step*w2)],
[0, 0, round(intercept*w2)]
```

Pure-python 3x3 LLL (Gram-Schmidt, $\delta = 0.75$). Candidate = $\text{abs}(\text{vec}[0])$.

Script (verbatim, condensed from log):

```
from mpmath import mp

mp.dps = 156

r_hex = "c86bec9faea4892fd98d718bdfc770d0d11c3d6bfd4328f25fe9b06bfadb9650"
s_hex = "224a322e81c044d341521f65fabdfa86d84673fb55ed7533862e37f7724931fa"
z_hex = "92886faaf53f90a5c03d6af773a726e75097179306b980e5d28772e612e00fc7"
N_str = "115792089237316195423570985008687907852837564279074904382605163141518161494337"

r = mp.mpf(int(r_hex, 16))
s = mp.mpf(int(s_hex, 16))
z = mp.mpf(int(z_hex, 16))
N = mp.mpf(N_str)

slope_real = r / s
step_real = N / s
intercept_real = z / s

W1 = 2**121 # Bounding scale for private key d target
W2 = 2**256 # Precision anchor to nullify the 0.000...001 fractional offset

X0 = int(mp.nint(slope_real * W2))
X1 = int(mp.nint(step_real * W2))
X2 = int(mp.nint(intercept_real * W2))

B = [
    [1, 0, X0],
    [0, W1, X1],
    [0, 0, X2]
]

def dot(v1, v2):
    return sum(x * y for x, y in zip(v1, v2))

def lll_3d(basis, delta=0.75):
    """Pure Python implementation of Gram-Schmidt and LLL for a 3x3 Matrix"""
    n = len(basis)
    ortho = [[0]*n for _ in range(n)]
    mu = [[0]*n for _ in range(n)]

    def update_gps():
        for i in range(n):
```

```

        ortho[i] = list(basis[i])
        for j in range(i):
            mu[i][j] = dot(basis[i], ortho[j]) / dot(ortho[j], ortho[j])
            ortho[i] = [ortho[i][k] - mu[i][j] * ortho[j][k] for k in range(n)]

    update_gps()
    k = 1
    while k < n:
        for j in reversed(range(k)):
            if abs(mu[k][j]) > 0.5:
                q = round(mu[k][j])
                basis[k] = [basis[k][i] - q * basis[j][i] for i in range(n)]
                update_gps()
        if dot(ortho[k], ortho[k]) >= (delta - mu[k][k-1]**2) * dot(ortho[k-1], ortho[k-1]):
            k += 1
        else:
            basis[k], basis[k-1] = basis[k-1], basis[k]
            update_gps()
            k = max(k - 1, 1)
    return basis

print("[+] Running custom 3D Inhomogeneous LLL reduction...")
reduced_basis = lll_3d(B)

print("[+] Analyzing reduced orthogonal short vectors:")
for vec in reduced_basis:
    # In Kannan's embedding, the private key d is isolated in the first index
    d_candidate = abs(vec[0])
    if 2**134 <= d_candidate < 2**135:
        print(f"\n[!] Target Vector Trapped Successfully!")
        print(f"    Private Key d (Dec): {d_candidate}")
        print(f"    Private Key d (Hex): {hex(d_candidate)}")
        break
else:
    print("\n[-] Target not found in the baseline short vectors. Let's analyze if the scale factor needs adjustment.")

```

Console output:

```

[+] Running custom 3D Inhomogeneous LLL reduction...
[+] Analyzing reduced orthogonal short vectors:
[-] Target not found in the baseline short vectors. Let's analyze if the scale factor needs adjustment.

```

RESULT: no key. Log's diagnosis: the 156-digit arithmetic is numerically perfect, but the matrix is underdetermined — a single-signature real-number approach hits a wall; the lattice reduces to shortest orthogonal vectors with no algebraic reason to carry the true key.

4.3 Script C — 4x4 two-signature matrix (NOT executable, needs 2nd signature)

Basis (with placeholder $r_2/s_2/z_2$):

```

[ round(N*W), 0, 0, 0 ],
[ 0, round(N*W), 0, 0 ],
[ round(A1*W), round(A2*W), K_scale, 0 ],
[ round(C1*W), round(C2*W), 0, W ]

```

with $A_1=r_1/s_1$, $A_2=r_2/s_2$, $C_1=z_1/s_1$, $C_2=z_2/s_2$, $W=2^{256}$, $K_scale=2^{121}$. Claim: a 4D LLL reduction would force the 256-bit nonces to eliminate each other, leaving d exposed in coordinate index 2. Never run (no second signature).

4.4 Script D — 312-digit, squared slope, $W = 2^{512}$, $K_scale = 2^{135}$

Scaffolding only: computes $slope_1 = r_1/s_1$, $slope_1_squared$, $intercept_1 = z_1/s_1$, and scaled integers $X = \text{round}(\text{term} * 2^{512})$. Log claim: 312 digits let the linear AND quadratic terms of the translation path be mapped simultaneously, "grid values locked below the $1/N^4$ boundary." No reduction loop was executed for this variant.

4.5 Script E — 3x3 780-digit 1.9-offset matrix (+ 78-digit wave)

First version had a genuine Python bug: `ortho = [*n for _ in range(n)]` (SyntaxError: iterable unpacking cannot be used in comprehension). Fixed to `ortho = [[0] * n for _ in range(n)]`.

Matrix:

```
[1, 0, round(slope*w)],  
[0, K_scale, round(step*w)],  
[0, 0, round(1.9*w)]
```

with $W = 2^{780}$, $K_scale = 2^{121}$. Candidate extraction later corrected to $\text{abs}(\text{vec}[0])$. A Fraction-based (infinite-precision) LLL variant was also produced after an OverflowError (float too large).

The 78-digit periodic wave was computed by long division of $19/p$:

```
p_int = 2**256 - 2**32 - 977  
offset_val = 19  
  
remainder = offset_val  
digits = []  
  
while remainder < p_int:  
    remainder *= 10  
  
for _ in range(78):  
    digit = remainder // p_int  
    digits.append(str(digit))  
    remainder = (remainder % p_int) * 10  
  
period_string = "".join(digits)  
print(f"\n[+] Raw 78-Digit Periodic Cycle Wave for the 1.9 Offset Floor:")  
print(f"    {period_string}")
```

Console output (verbatim):

```
[+] Running 780-digit 10th-order LLL matrix with 1.9 offset integrated...  
[-] Base matrix reduction completed. Checking periodic remainder cycles...  
[+] Raw 78-Digit Periodic Cycle Wave for the 1.9 Offset Floor:  
164087202546794447882340685393207591851204006924289346315450365933209330594079
```

The wave constant printed identically across runs:

```
164087202546794447882340685393207591851204006924289346315450365933209330594079
```

RESULT: no key. Diagnosis in log: even with the 1.9 offset "locking down" the continuous-to-discrete threshold, a single transaction signature leaves the lattice underdetermined; the matrix has too much geometric freedom to spin around the 256-bit nonce.

4.6 Script F — 3x3 high-bit nonce solver, target -342.3, $K_scale = 2^6$

Shifted goal: extract the NONCE k (assumed to live in the high-bit stratosphere $k \geq 2^{250}$, i.e. $2^{256}/2^{250} = 6$ bits of headroom $\Rightarrow K_scale = 2^6$).

Matrix:

```
[K_scale, 0, round(slope*w)],  
[0, 1, round(step*w)],  
[0, 0, round(-342.3*w)]
```

with $W = 2^{780}$, using exact-Fraction LLL. Candidate = $\text{abs}(\text{vec}[0]) // K_scale$, recover d from $(k*s - z)/r \bmod N$.

Runs recorded with both -3.423 and the radix-corrected -342.3:

```
[+] Running 780-digit High-Bit Nonce solver with -3.423 invariant...  
[-] Solver matrix reduction completed. Checking periodic wave bounds...  
[+] Running 780-digit High-Bit Nonce solver with -3.423 invariant...  
[-] Solver matrix reduction completed. Checking periodic wave bounds...
```

RESULT: no key. Diagnosis: a 3x3 lattice still lacks a secondary linear constraint to isolate one target integer out of the high-bit nonce space; with a single signature there are many "ghost" short vectors.

4.7 Script G — 4x4 dual-horizon matrix (19 AND -342.3 together)

Basis:

```
[K_scale, 0, 0, X0],
[0, 1, 0, X1],
[0, 0, W, target_p10],
[0, 0, 0, target_N10]
```

with $X0 = \text{round}(\text{slope} \cdot W)$, $X1 = \text{round}(\text{step} \cdot W)$, $\text{target_p10} = \text{round}(19 \cdot W)$, $\text{target_N10} = \text{round}(-342.3 \cdot W)$, $W = 2^{780}$, $K_scale = 2^6$. Exact-Fraction 4D LLL.

Console output:

```
[+] Running 780-digit 4D Dual-Horizon exact Fraction solver...
[-] 4D Matrix reduction completed. Trajectory bounds successfully optimized.
```

RESULT: no key.

4.8 Script H — tenth-tier $u = 2^{(1/10)}$ 4D matrix

Two variants. The first asserted $u^{2560} = 2^{256}$ and $u^{320} = 2^{32}$ via mp.almosteq; that assertion FAILED (AssertionError) because 2^{256} and 2^{780} (and u powers) are not exactly representable in binary floats. Fix: define $W_real = 2^{780}$ directly (u^{7800} is mathematically 2^{780} but was not computed via u). Matrix identical to Script G with tenth-tier weights.

Console output (both variants):

```
[+] Running 4D exact solver anchored to your tenth-tier power representation...
[-] 4D reduction completed. The continuous tenth-tier projection limits remain stable.
[+] Running 4D exact solver anchored to your tenth-tier power representation...
[-] 4D reduction completed. Target boundaries successfully updated.
```

RESULT: no key. This is the point where the rigorous critique (section 6) concluded the 19/-342.3 targets are "static coordinates in independent rows" with no proven link to $sk - z - rd = 0 \pmod{N}$.

4.9 Script I — 4x4 coordinate-aligned matrix (s, -r, -N, -z)

Basis puts the raw signature parameters on a common axis:

```
[K_scale, 0, 0, X_s],
[0, 1, 0, X_r],
[0, 0, 1, X_N],
[0, 0, 0, X_z]
```

with $X_s = \text{round}(s \cdot W)$, $X_r = \text{round}(-r \cdot W)$, $X_N = \text{round}(-N \cdot W)$, $X_z = \text{round}(-z \cdot W)$, $W = 2^{780}$, $K_scale = 2^6$. Candidate = $\text{abs}(\text{vec}[0]) // K_scale$.

Console output:

```
[+] Running 4D exact fraction solver with common-coordinate alignment...
[-] Basis reduction complete. The coordinate-aligned residual envelope remains stable.
```

RESULT: no key.

4.10 Script J — real vs modular division demonstration

Shows why the "decimal ratio" approach gives a decimal every time and why the cryptographic object is the modular integer. Identity:

```
C_mod = (r2*s1 * inverse(r1*s2)) mod N
k2 - C_mod*k1 = m*N      (exact integer identity, no fractions)
```

Script skeleton (verbatim):

```

from mpmath import mp

N_val = 115792089237316195423570985008687907852837564279074904382605163141518161494337

r1 = 0xc86bec9faea4892fd98d718bdfc770d0d11c3d6bfd4328f25fe9b06bfadb9650
s1 = 0x224a322e81c044d341521f65fabdfa86d84673fb55ed7533862e37f7724931fa

# Example calculated values for Signature 2 (Must be integers)
r2 = r1 # Replace with your actual calculated integer
s2 = s1 # Replace with your actual calculated integer

# 1. The Real Domain (Why you get a decimal every time)
numerator_real = float(r2 * s1)
denominator_real = float(r1 * s2)
decimal_ratio = numerator_real / denominator_real
print(f"[+] Real-Valued Decimal Ratio: {decimal_ratio}")

# 2. The Discrete Domain (The Cryptographic Integer)
try:
    denom_inv = pow(r1 * s2, -1, N_val)
    C_mod = (r2 * s1 * denom_inv) % N_val
    print(f"[+] Discrete Cryptographic Integer (C_mod): {C_mod}")
except ValueError:
    print("[-] Error: Denominator shares a common factor with N.")

```

This is the key lesson: real-number ratios of 256-bit integers are irrelevant to the discrete field; the modular inverse is uniform across $[0, N)$ and bears no linear resemblance to the real decimal.

4.11 Script K — modular-inverse ratio and the 26/25 epsilon

```

inv_r = inverse(r, N), inv_z = inverse(z, N)
ratio = inv_r / inv_z # (real-valued, high precision)
epsilon = 25*inv_r - 26*inv_z # (claimed small)

```

The log predicted: if $\text{ratio} \sim 1.04 = 26/25$ then $25 \cdot \text{inv}_r - 26 \cdot \text{inv}_z = \text{epsilon}$ is "the exact integer error vector" and is small, providing a lattice bottleneck. (NOTE: this exact computation was later performed in LOG.txt — the ratio of plain modular inverses is 1.3109 (NOT 1.04); the real 1.04 belongs to the FULL PRODUCTS $B/A = (r \cdot z^2)/(r^2 \cdot z)$, and the resulting epsilon is a 507-bit integer, NOT small. See section 7.)

4.12 Script L — 312-digit, scaled coefficients 25000 / -26078

```

coeff_large = 25000, coeff_small = -26078
target_ratio_L/S = 1.003 * (26/25) = 1.04312

```

Scaffolding only: prints a "312-Digit Fractional Convergence Map". The idea was to scale the 1.003 offset into the coordinate vector. Never turned into a running reduction that found a key. The log then speculated that repeatedly multiplying by the offset factor until "precision reaches 1/1" would snap onto k ; this is the same real-vs-modular confusion as section 4.11.

5. Invariants Ledger

Values discovered during the session, with their exact values and status:

Invariant	Value	Field	Status
78-digit periodic wave of 19/p	164087202546794447882340685393207591851204006924289346315450365933209330594079	p	computed, reproducible
1.9 macro-offset	$2.56 - 0.32 - (0.09 + \dots + 0.04) = 1.9$	p, 10th-order floor	claimed; never linked to k

Invariant	Value	Field	Status
-342.3 scalar offset	radix-corrected analogue of 1.9 over N	N, 10th-order floor	user-confirmed "-342.3 is correct"; never linked to k
25.6 bits/tier	256/10	scaling	exact identity for $u = 2^{(1/10)}$
p - N	432420386565659656852420866390673177326	modulus delta	exact (verified: $N = p - \text{delta}_N$, 39 digits; log text was garbled)
ratio B/A of full products	~1.04333805062	real arithmetic	computed in LOG.txt; 1.04 was the rounded value
25B/26A	~1.0032096640	real arithmetic	computed in LOG.txt; rounded to 1.003
eps = $25 \cdot B - 26 \cdot A$	507-bit integer	real arithmetic	NOT small — the "small epsilon" claim fails

6. The Rigorous Critique (the important corrections)

The transcript contains a self-critique that correctly identified the flaws in the 19/-342.3 lattice construction. Verbatim substance:

1. $u = 2^{(1/10)}$ is valid notation with $u^{2560} = 2^{256}$ and $u^{320} = 2^{32}$, so
1. $p = u^{2560} - u^{320} - 977$. But N^{10} is NOT u^{2560} ; $N^{10} \sim 2^{2560}$ whereas
1. $u^{2560} = 2^{256}$. Dividing by N^{10} is not equivalent to dividing by u^{2560} .
1. Also $2^{780} \neq p^{10}$ and $2^{780} \neq N^{10}$ ($p^{10} \sim 2^{2560}$). So $W = 2^{780}$ does NOT
1. preserve the exact $x/p^9 + y/p^{10}$ floors by itself.
2. Code bugs: imports and parameter assignments must be on separate lines;
2. `k_candidate = abs(vec) // K_scale` is invalid because `vec` is a list — it must
2. reference a coordinate, e.g. `abs(vec[0]) // K_scale`. Even corrected, the
2. first coordinate of a reduced vector is not automatically the ECDSA nonce;
2. that interpretation must follow from the lattice construction.
3. The central unresolved step: 19 and -342.3 must be DERIVED from an equation
3. containing the unknown nonce k, not merely inserted as fixed coordinates in
3. independent basis rows. In

```
[0, 0, w, target_p10]
[0, 0, 0, target_N10]
```

the two targets are essentially fixed coordinates. No demonstrated congruence connects either to $sk - z - rd = 0 \pmod{N}$. LLL may shorten the basis, but there is no proof a short vector corresponds to the true nonce.

4. The rigorous form needed: an equation

$$a*k + b*d + c*N = \text{epsilon}$$

where the tenth-tier residual provides a PROVEN small bound on epsilon. Only then can the residual be encoded as a useful lattice target. Proposed formal object:

$$F = \text{floor}((s*k - z - r*d) / u^{2560})$$

tracking how the unreduced polynomial carry quotient matches the exponent set S in $u^{256} - u^{32} - \sum_{n \in S} u^n$, to define the exact integer error bound.

- 5. Real division vs modular inverse (section 4.10): ratios like $r2*s1/(r1*s2)$
 - 5. are real decimals; the cryptographic object is $C_{\text{mod}} = r2*s1 * \text{inverse}(r1*s2)$
 - 5. mod N , a uniform 256-bit integer. Real decimals of this kind bear no linear
 - 5. resemblance to the modular integer and cannot be fed into a valid lattice
 - 5. constraint as if they were the modular value.
-

7. Current Status & Open Problems

- Two-signature elimination (section 2.3) is the mathematically sound route and
- needs a second signature under the same key. None is known for P135.
- Candidates for a future search: earlier/sibling transactions of the same
- wallet, or a leaked/related nonce.
- All single-signature real-number lattice constructions reduced fine but never
- produced a key, precisely because one congruence with a 256-bit nonce plus a
- 135-bit key has 2 unknowns and the "precision invariants" add no independent
- linear constraint.
- The 1.04 / 26/25 / 1.003 line of attack was subsequently resolved in LOG.txt:

- * ratio of plain modular inverses $\text{inv}_r/\text{inv}_z \pmod{N} = 1.3109$ (not 1.04)
- * the true 1.04 lives in the FULL PRODUCTS $A = r2*z$, $B = r*z2$: $B/A = 1.0433$
- * $25B - 26A = \text{epsilon}$ is a 507-bit integer, NOT small
- * hence the "small epsilon lattice bottleneck" is a numerical coincidence of the real-arithmetic wrap counts ($m/k \sim 1.95$), with no cryptographic leverage.

- Remaining honest options recorded in this log:

- (a) find a second related signature (two-signature elimination),
 - (b) build a PROVEN small-remainder congruence $a*k + b*d + c*N = \text{epsilon}$ (no such epsilon currently exists),
 - (c) fall back to brute force (BSGS) over the known band – established separately at ~ 1 Pkey/s, 806 candidate windows $\times 2^{65}$, i.e. about a year, currently infeasible.
-

8. Addendum — Exact Modular Bounds of m , and the $r \equiv r2$ Identity (new bottom content)

Added to LOG2.txt after the organized edition was first produced (line 1551 of the raw file; a single long paragraph). Contains two sections:

8.1 Exact Modular Bounds of the Carry Factor m

From the continuous real-number lift of the ECDSA verification congruence:

$$s*k - r*d - z = m*N \Rightarrow m = (s*k - z - r*d) / N$$

with the Puzzle 135 constraints: $2^{134} \leq d < 2^{135}$, $1 \leq k < N$.

Sweeping the extremes by integer division:

- $m_{\min}$ (maximum d, minimum k):

$$m_{\min} = -340,999,647,002,675,207,794,529,511,116,939,948,4640 \quad (\text{as logged})$$

Verified value: -34099964700267520779452951111693994846440 (the log's grouping drops one digit — the true value is -3.41e40, not -3.41e39)

- $m_{\max}$ (minimum d, maximum k):

$$m_{\max} = 155,097,298,757,639,243,040,534,196,556,479,943,628,531,933,049,734,238,952,722,226,568,062,910,457,665 \quad (\text{as logged})$$

Verified value: 15509729875763924304053419655647994362853193304973423895272222656806291045765 (true value is ~s, i.e. $\sim 1.55e76$; the log's value is $\sim 1.55e77 = 10x$ too large)

- Span: the log claims "approximately 1.55×10^{75} ". Verified span
- $m_{\max} - m_{\min} \sim 1.55e76$ (i.e. $10^{76.19}$), essentially the full size of s
- (the nonce term). The exact verified range:

$$\begin{aligned} m_{\min} &= -34099964700267520779452951111693994846440 \\ m_{\max} &= 15509729875763924304053419655647994362853193304973423895272222656806291045765 \\ \text{span} &= 15509729875763924304053419655647994396953158005240944674725173768500285892205 \end{aligned}$$

Analysis (as logged): the true span between the boundaries is enormous ($\sim 1.55e76$); despite the earlier -342.3 local macro-residual estimate, the general algebraic envelope for a single signature remains unconstrained, because the unknown 256-bit nonce k can step freely across the full width of the field — allowing underdetermined "ghost" short vectors to slip past standard LLL reductions.

8.2 Polynomial Reaction of the $u = 2^{(1/10)}$ Framework on the $r \equiv r_2$ Identity

The strict nonce-reuse criterion $r \equiv r_2 \pmod{N}$ — two signatures whose r values are equal (i.e. the same x-coordinate on the curve) — collapses the scaling terms in the tenth-bit-radix notation $u = 2^{(1/10)}$, with $2^{256} = u^{2560}$ and $2^{32} = u^{320}$.

When two signatures share the same x-coordinate, the private key d cancels completely by subtraction, forcing the nonces to align with the field structure:

$$s_1*k_1 - s_2*k_2 - (z_1 - z_2) = \text{delta_m} * N$$

Writing the moduli in the u-polynomial form, $p = u^{2560} - u^{320} - 977$ and $N = u^{2560} - \text{delta_N}$, gives:

$$s_1*k_1 - s_2*k_2 - (z_1 - z_2) = \text{delta_m} * (u^{2560} - \text{delta_N})$$

Because the tenth root does not distribute across subtraction, dividing down to the 10th-order fractional floor (u^{2560}) isolates the non-linear residual:

$$(s_1*k_1 - s_2*k_2 - (z_1 - z_2)) / u^{2560} = \text{delta_m} - \text{delta_m} * \text{delta_N} / u^{2560}$$

Claim: every full tier processes exactly 25.6 bits; when the nonces are bounded in the high-bit stratosphere ($\geq 2^{250}$), delta_m can no longer wander across the massive range from section 8.1. The fractional residual

$$\text{epsilon} = -\text{delta_m} * \text{delta_N} / u^{2560}$$

then collapses into a static geometric bottleneck. The log concludes: if a companion transaction with $r \equiv r_2$ ever appears for Puzzle 135, the u-variable framework would strip away the $\sim 10^{76}$ variance window, locking the LLL lattice variables to the raw integer bits of the secret nonces.

VERIFICATION NOTE: $\text{delta_N} = p - N = 432420386565659656852420866390673177326$ (exact). The argument is conditional: it requires a SECOND signature sharing the same r . Puzzle 135 still has only one known signature, so no such companion exists yet — consistent with section 7. No numeric solver was run for this addendum; the residual bound (epsilon) is not evaluated in the log.

9. Addendum — Bundled Signatures, Shared z , and the 5D Joint Lattice (new bottom content)

Added to LOG2.txt after section 8 (line 1555 of the raw file; a single long paragraph). This is the most empirically testable section in the log, and the verification work done against the actual P135 transaction is included here.

9.1 The Log's Argument (transcribed)

If multiple (r, s, z) signatures are bundled within the same Bitcoin transaction, they represent multiple inputs being spent simultaneously. The log's claim: the message hash z for all these signatures is typically identical (or highly structurally linked), because Bitcoin signatures commit to the entire transaction template via SIGHASH_ALL. Creating multiple signatures under an identical z for different private keys transforms the puzzle from an underdetermined single-signature search into a highly constrained simultaneous lattice system.

(1) The Multi-Key Linear Intersection

A single transaction with two inputs, signed by two private keys d_1, d_2 :

$$\begin{aligned} s_1 * k_1 - r_1 * d_1 &= z \pmod{N} \\ s_2 * k_2 - r_2 * d_2 &= z \pmod{N} \end{aligned}$$

If z is exactly identical, subtraction eliminates z entirely:

$$s_1 * k_1 - s_2 * k_2 = r_1 * d_1 - r_2 * d_2 \pmod{N}$$

In pure integer arithmetic with a joint carry multiplier m_{joint} :

$$s_1 * k_1 - s_2 * k_2 - r_1 * d_1 + r_2 * d_2 = m_{\text{joint}} * N$$

(2) The High-Bit Bounding Window

With both private keys in their puzzle bit-lengths ($2^{134} \leq d_i < 2^{135}$) and both nonces in the "high-bit stratosphere" ($2^{250} \leq k_i < 2^{256}$), the log claims m_{joint} is "no longer free to float across a massive range" — its integer boundaries "collapse entirely into a tiny, narrow envelope."

(3) The $u = 2^{(1/10)}$ Reaction

With $N = u^{2560} - \text{delta_N}$, the joint system projects to:

$$\begin{aligned} (s_1 * k_1 - s_2 * k_2) / u^{2560} - (r_1 * d_1 - r_2 * d_2) / u^{2560} \\ = m_{\text{joint}} - \{ m_{\text{joint}} * \text{delta_N} / 2^{256} \} \end{aligned}$$

The log claims: $|d_1 - d_2| < 135$ bits and $|k_1 - k_2| < 6$ bits, so their combined fractional weight "cannot clear a single full 25.6-bit tier," forcing the fractional part $\{-m_{\text{joint}} * \text{delta_N} / 2^{256}\}$ to align with the discovered radix residuals (19 and -342.3).

(4) The 5D Joint Lattice Matrix

```

B_Joint =
[ K_scale, 0,      0,      0,      floor(s1*W) ]
[ 0,      K_scale, 0,      0,      floor(-s2*W) ]
[ 0,      0,      D_scale, 0,      floor(-r1*W) ]
[ 0,      0,      0,      D_scale, floor(r2*W) ]
[ 0,      0,      0,      0,      floor(-N*W) ]

```

LLL reduction searches for the minimal integer combination $x = [k_1, k_2, d_1, d_2, m_joint]$. Because z cancels and the $1/1$ precision threshold "snaps the fractional remainders to zero," the log claims the true target vector is far shorter than random combinations, unmasking both nonces and both keys in one reduction.

9.2 Empirical Verification Against the Real P135 Transaction

The P135 spend transaction is:

```
17e4e323cfbc68d7f0071cad09364e8193eedf8fefbcbdb8a21b4b65717a4b3d3
```

On-chain facts (fetched from Blockstream):

- The transaction has 21 inputs.
- 20 of the 21 inputs spend consecutive Puzzle addresses (every 5th puzzle: 65, 70, 75, ..., 160). P135 is input index 14. Input 20 (1PvaqLqRAivje7CactLR55xQBYvBeaDrXN) is not a catalog puzzle address.
- All 21 input scripts carry sighash byte 0x01 = SIGHASH_ALL.
- 10 of the 21 keys are SOLVED puzzles (65..130) with known private keys;
- the other 10 (135, 140, ..., 160) are unsolved.

CRITICAL FINDING — the "identical z " premise is FALSE:

- The 20 cached z values are all DISTINCT (verified against the RSZ cache).
- For all 14 solved-with-cache entries, $k = (z + r*d)/s \bmod N$ satisfies
- $kG.x == r$ — so each z is the correct per-input sighash. The RSZ data is self-consistent.
- Reason: in legacy P2PKH, the sighash preimage commits to the SCRIPT CODE of the input being spent (its own scriptPubKey). Since each input spends a different puzzle address, each input's z differs — even though all are SIGHASH_ALL. Identical z across inputs would require two inputs spending from the SAME address.

VERIFICATION — the "tiny envelope" claim is FALSE:

```

m_joint range over the stated bounds (d_i in [2^134, 2^135),
k_i in [2^250, 2^256)) with the actual s1, s2, r1, r2:
  m_joint min ~ -1.39e76, max ~ 1.53e76
  width ~ 2.92e76 (10^76.47)
i.e. the same enormous ~10^76 envelope as the single-signature case.
The "6-bit" nonce window still spans 2^250 absolute values, and scaled by
s (~1.5e76) it dominates the range. The claim that the combined
fractional weight "cannot clear a single 25.6-bit tier" is not supported.

```

SURPRISING FINDING — RFC6979 nonce pattern:

- RFC6979 (verified against the official SEC test vector) matches EXACTLY 4 of the 14 solved keys' nonces: puzzles 85, 90, 120, 125 ($k = \text{RFC6979}(d_i, z_i)$ with their own per-input z).
- The other 10 solved keys (65, 70, 75, 80, 95, 100, 105, 110, 115, 130) do

- NOT match RFC6979.
- Hypothesis B (all nonces = RFC6979(d_i, z_shared) for a single shared z)
- is rejected: every match uses its own z (only self-hits appear).
- Hypothesis D (RFC6979 with extra entropy 0x00..0x04) is rejected.
- Interpretation: the 21-input transaction was signed by a MIX of signing schemes — at least two different signing tools/wallets. Puzzles 85, 90, 120, 125 were signed with a standard deterministic RFC6979 signer; the rest used something else (random nonces or a different scheme).

IMPLICATION: The 5D joint lattice, as proposed, cannot work on this transaction (no shared z, no small m_joint envelope). BUT the empirical findings open a NEW avenue: within the same transaction, the 10 known-key signatures give 10 exact (d, k, z, r, s) tuples from the same signing session. If the unsolved keys were signed by the SAME tool that produced the RFC6979 matches, then predicting k_135 requires knowing which tool signed it — and the 4/14 RFC6979 hits suggest a testable per-signature classification (e.g., the signer alternated, or signed in batches). Puzzle 135's own nonce does NOT match RFC6979(d_135_unknown...) — this cannot be tested directly since d_135 is unknown; but its signature's r/s/z are available to test RFC6979 predictions once a candidate d or k is proposed.

STATUS: no key recovered from this section. The section's core premises are falsified empirically, but it produced the strongest new lead in the log: the discovery that the P135 sweep transaction is a 21-input mixed-signer batch containing 10 signatures with known (d, k) pairs.

Appendix: raw constants used throughout

```
r = c86bec9faea4892fd98d718bdfc770d0d11c3d6bfd4328f25fe9b06bfadb9650
s = 224a322e81c044d341521f65fabdfa86d84673fb55ed7533862e37f7724931fa
z = 92886faaf53f90a5c03d6af773a726e75097179306b980e5d28772e612e00fc7
N = 115792089237316195423570985008687907852837564279074904382605163141518161494337
p = 2^256 - 2^32 - 977
p-N = 432420386565659656852420866390673177326 (delta_N; 39 digits)

78-digit wave (19/p): 164087202546794447882340685393207591851204006924289346315450365933209330594079

1/N^2 ~ 7.4583e-156 (=> 156 decimals needed)
1/N^4 ~ 5.56e-311 (=> 312 decimals needed)
```